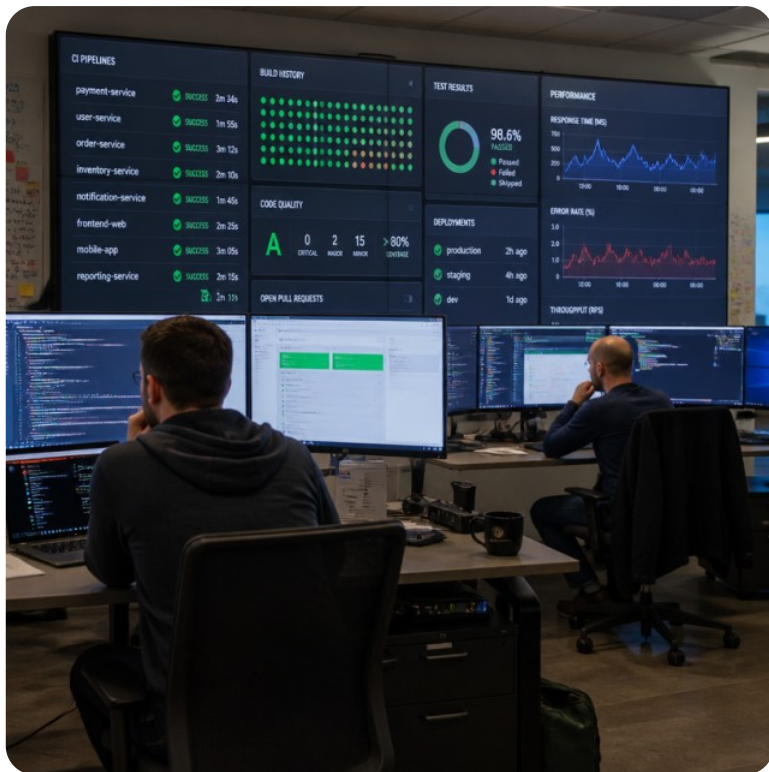


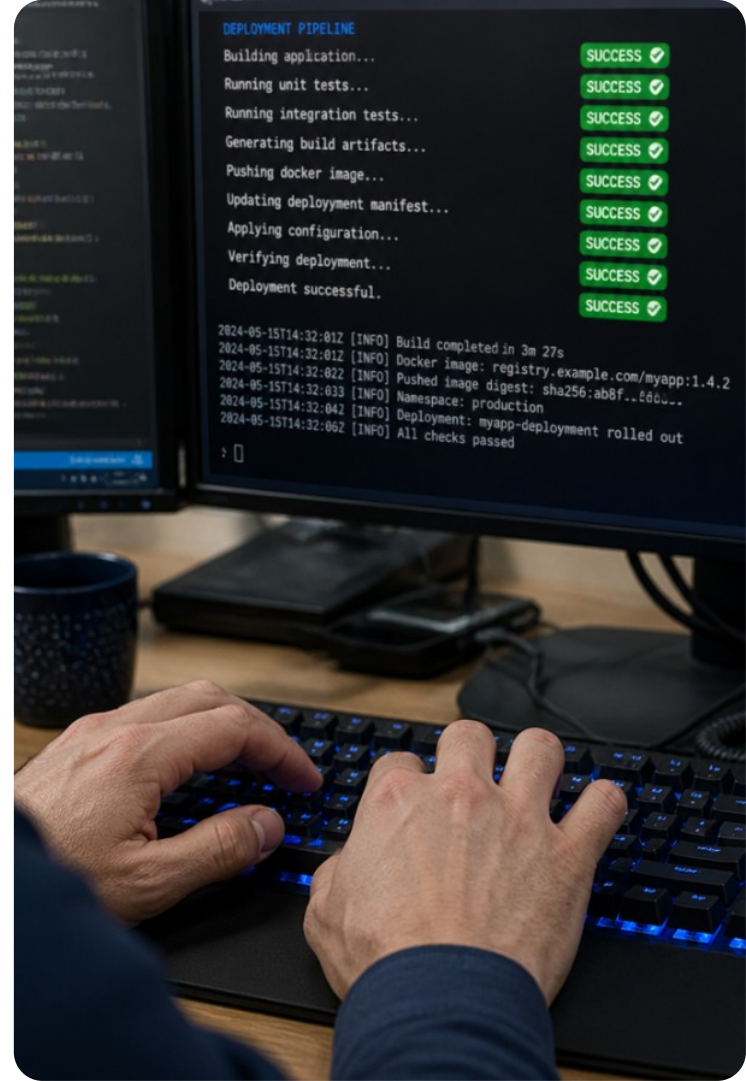


Qualidade, Performance, Acessibilidade e CI/CD

Automação contínua, testes não-funcionais e entrega confiável no frontend.

O Dilema da Sexta-Feira

Você confiaria em publicar um software crítico para milhares de usuários com apenas um clique?



Objetivos da Aula

O que vamos dominar hoje

Nesta nona etapa da nossa jornada, você aprenderá a:

Estruturar **pipelines de CI/CD** automatizadas com GitHub Actions.

Implementar auditorias de **acessibilidade (WCAG)** com axe-core.

Medir **Core Web Vitals** com Lighthouse e proteger merges.



Ponto-chave

Qualidade não é um evento final: é um processo sistemático e contínuo.



Vocabulário Essencial



Pipeline

Sequência automatizada de etapas que compila, valida e entrega software.



Workflow

Arquivo de configuração estruturado que define gatilhos, jobs e passos.



Desempenho

Velocidade de renderização, tempo de resposta e eficiência de recursos web.



Integração

Prática de unir alterações frequentes de código em um repositório central.

Recapitulação: O Ciclo TDD



REVISÃO TÉCNICA

Red-Green-Refactor

Ciclo TDD:

Red: Criar teste que falha.

Green: Implementar o mínimo para passar.

Refactor: Melhorar código mantendo testes.



Lembre-se

Ciclo garante código sempre testado.

Depuração Estratégica em Testes

REVISÃO TÉCNICA

Métodos de Debugging

Falhas exigem análise:

VS Code Breakpoints: Inspecciona escopo.

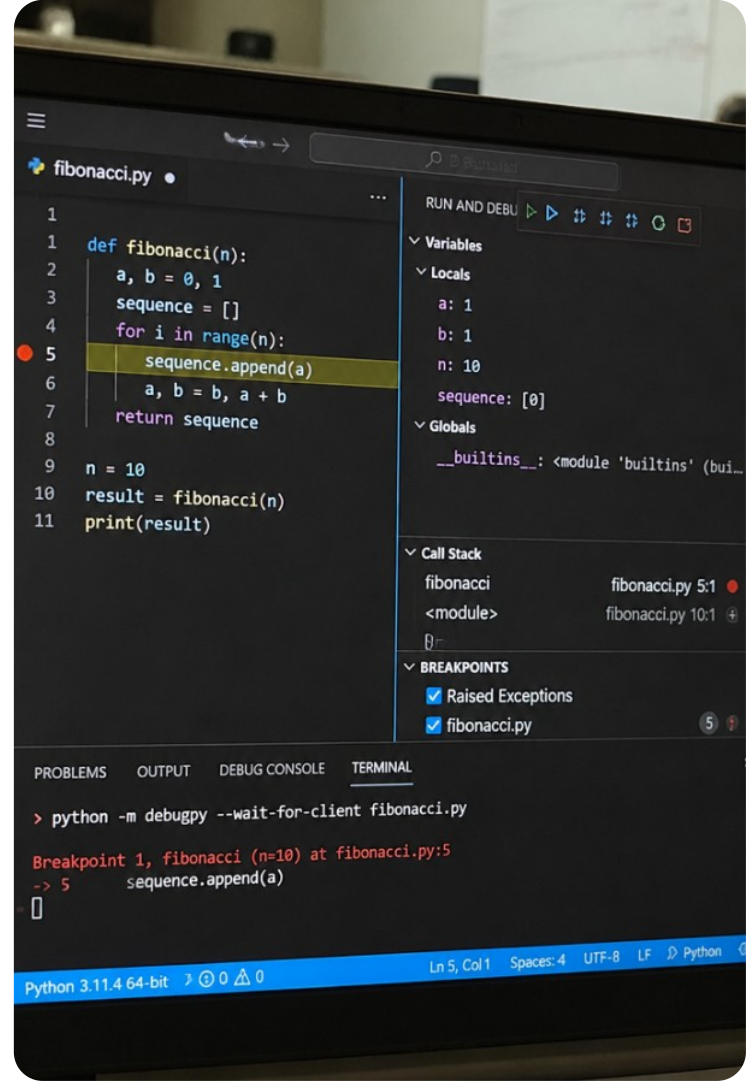
screen.debug(): Exibe DOM no terminal.

Time-travel: Reproduz estados passo a passo.



Atenção

Evite logs; use ferramentas de inspeção.



Quiz: TDD e Depuração



Pergunta 1:

Qual é a primeira ação obrigatória no ciclo clássico de TDD?

Pergunta 2:

Qual comando da Testing Library imprime o estado atual do DOM no terminal para debugging?

Pergunta 3:

O que deve ser preservado rigorosamente durante a fase de Refatoração?

Quiz: TDD e Depuração



Resposta 1:

Escrever um teste automatizado que falhe (fase Red) descrevendo o comportamento esperado.

Resposta 2:

```
screen.debug()
```

Resposta 3:

O comportamento funcional existente e todos os testes passando em verde.

O Conceito de CI/CD

CONCEITOS

Integração e Entrega Contínuas

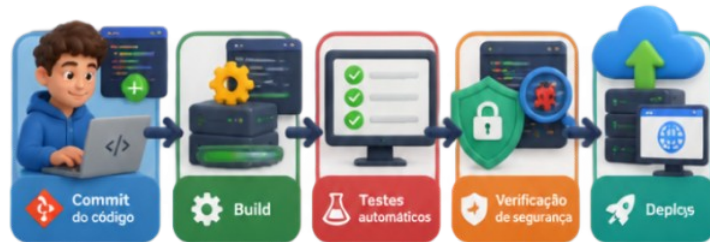
CI: Unifica o trabalho dos devs diariamente. Cada push dispara testes automáticos para validar o código.

CD: Garante que o software validado seja lançado rapidamente, de forma previsível e sem intervenção manual.

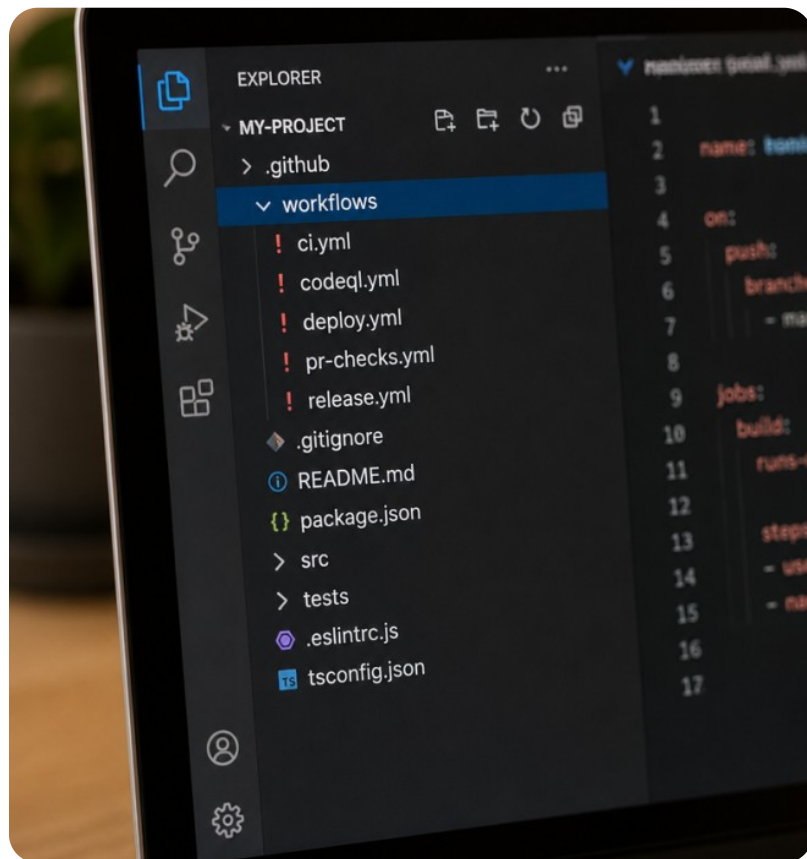


Ponto-chave

Automação de pipeline torna a qualidade um guardião permanente.



Anatomia do GitHub Actions



CONCEITOS

Componentes Estruturais

Automações nativas no repositório:

Workflows: YAML em .

Events: Triggers como , .

Jobs: Etapas em máquinas virtuais.

Steps: Comandos shell ou ações (ex:).



Exemplo

Workflow dispara testes unitários automaticamente a cada push ou PR.

Sintaxe do Arquivo YAML

Definição de Gatilhos

O workflow define quando deve rodar e qual ambiente preparar:

- : Nome legível da pipeline.
- : Escuta e na branch .
- : Especifica o SO ().

Passos de Execução

Sequência direta de tarefas no container:

- : Baixa o código.
- : Configura Node.js.
- : Instalação limpa e reproduzível.
- : Executa os testes.



Lembre-se

Use npm ci em servidores de integração contínua para respeitar fielmente o package-lock.json.

Validação em Push e Pull Requests

AUTOMAÇÃO

Feedback Rápido ao Desenvolvedor

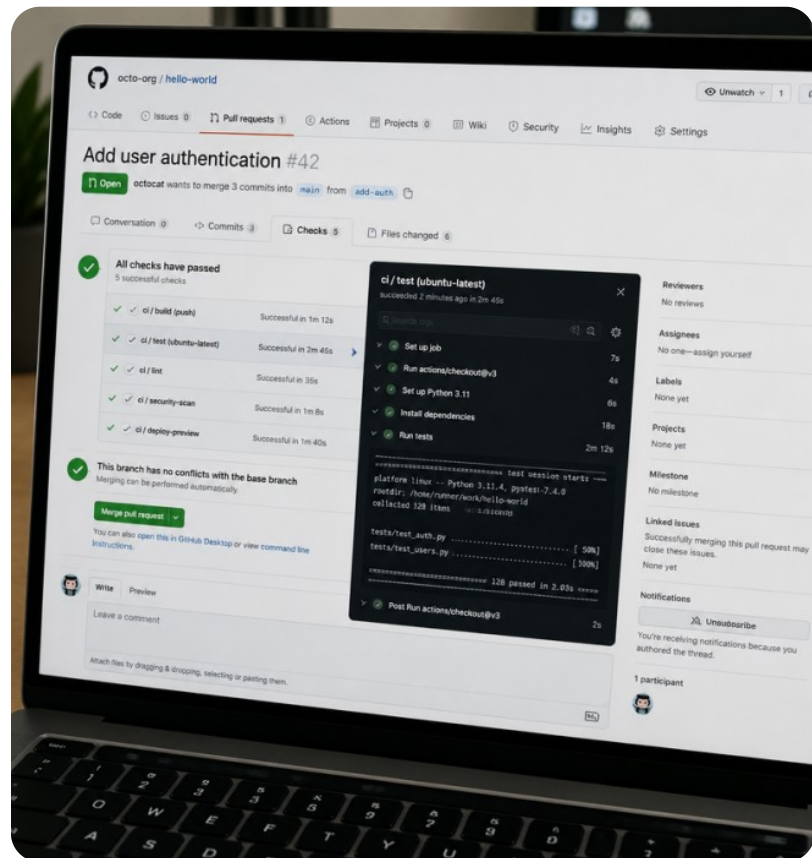
Ao abrir um PR, o CI clona o projeto, roda Vitest/Testing Library e exibe o veredito no GitHub.

- Testes aprovados: PR ganha selo verde.
- Falhas: O log aponta o teste, linha e erro.



Curiosidade

Equipes detectam 95% das quebras antes da revisão humana.



Bloqueio de Merge Automatizado

AUTOMAÇÃO

Branch Protection Rules

Regras que impedem merge de código falho:

Status checks: Bloqueia se testes falharem.

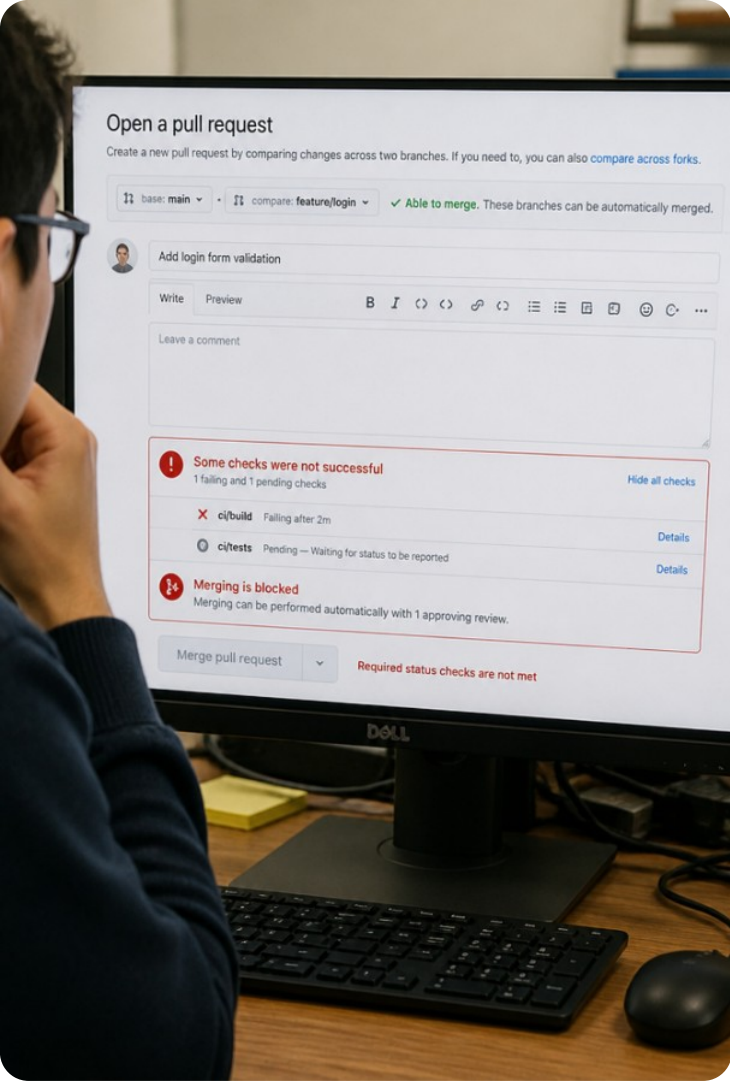
Up to date: Exige branch sincronizada com main.

Coverage Threshold: Rejeita PRs abaixo da meta.



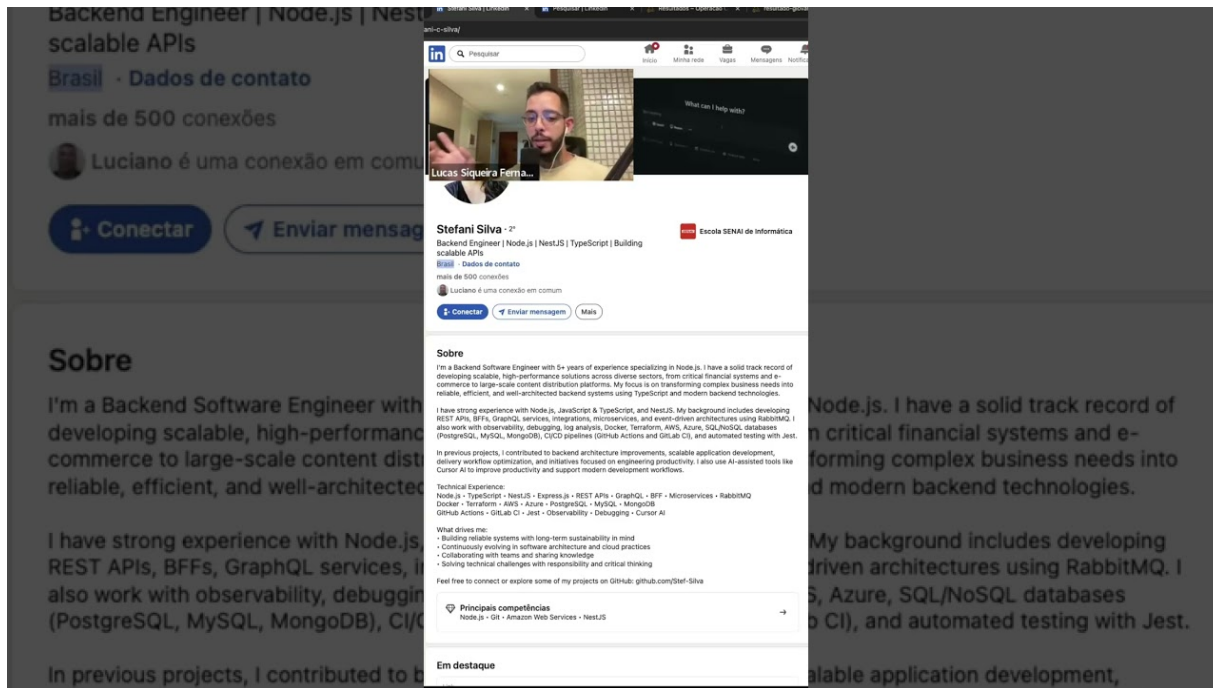
Atenção

Sem proteção, deploys com bugs ocorrem.



CI/CD em Ação no Desenvolvimento Web

Observe como os pipelines de CI/CD garantem testes contínuos e publicações seguras em ambientes reais de desenvolvimento web:



Verificação: Gatilhos e Regras

Qual configuração garante que um Pull Request não seja integrado caso os testes automatizados falhem?

1. Adicionar um comentário manual no pull request solicitando correção.
2. Excluir o arquivo de workflow para evitar que erros sejam notificados.
3. Branch Protection Rules exigindo aprovação dos status checks da CI.
4. Configurar o comando git commit com a flag --no-verify no terminal.

Verificação: Gatilhos e Regras



Qual configuração garante que um Pull Request não seja integrado caso os testes automatizados falhem?

1. Adicionar um comentário manual no pull request solicitando correção.
2. Excluir o arquivo de workflow para evitar que erros sejam notificados.
3. Branch Protection Rules exigindo aprovação dos status checks da CI.
4. Configurar o comando git commit com a flag --no-verify no terminal.

Além dos Testes Funcionais

O software funciona conforme a regra de negócio.
Mas ele é rápido e utilizável por todas as pessoas?



Acessibilidade Web e Diretrizes WCAG

ACESSIBILIDADE

Diretrizes WCAG 2.1

Web acessível garante inclusão:

Perceptível: Contraste, textos alt.

Operável: Teclado, foco visível.

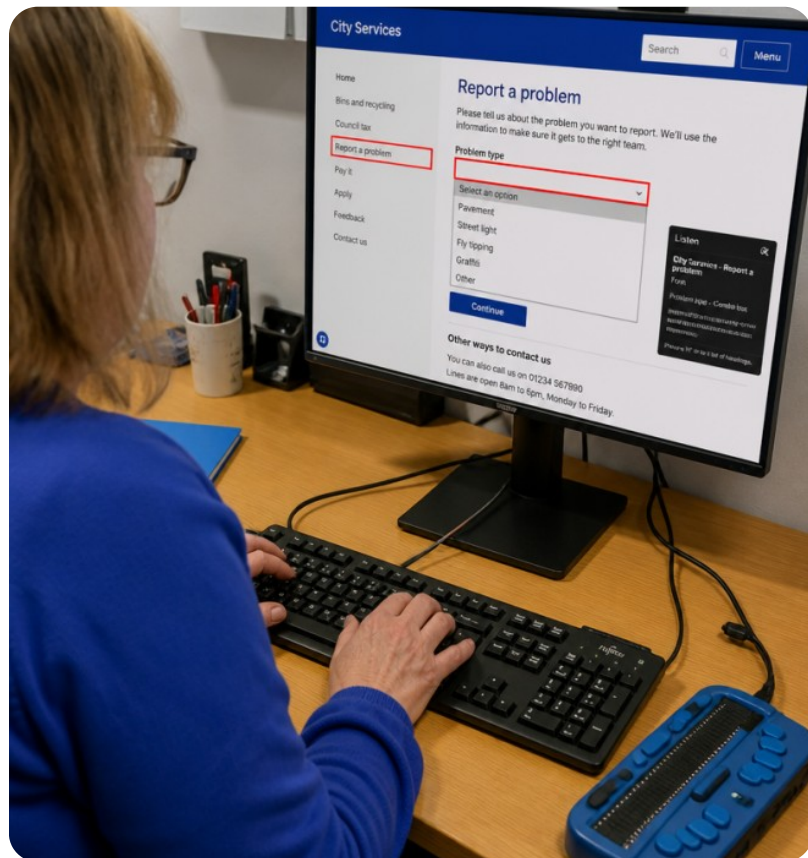
Compreensível: Linguagem clara.

Robusto: Tecnologias assistivas.

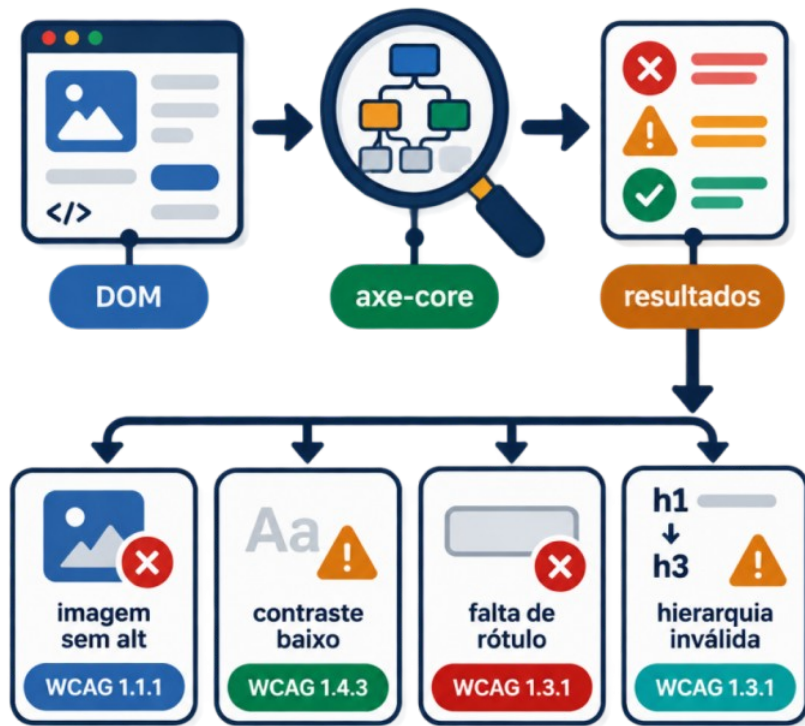


Ponto-chave

Acessibilidade: direito e requisito.



Testes Automatizados com axe-core



ACESSIBILIDADE

Auditoria no Vitest

O **axe-core** inspeciona o DOM, detectando falhas WCAG:

Imagens sem .

- Contraste baixo.

Falta de .

- Hierarquia de cabeçalhos inválida.

Exemplo

Use: `expect(await axe(container)).toHaveNoViolations()`.

Análise Crítica: Auditoria de A11y

Ferramentas automatizadas como o axe-core encontram 100% dos problemas de acessibilidade de uma página.



**VERDADE
IRO**



FALSO



Prepare-se para explicar o seu raciocínio.

Análise Crítica: Auditoria de A11y



Ferramentas automatizadas como o axe-core encontram 100% dos problemas de acessibilidade de uma página.



Por que é isso?

Automações detectam cerca de 30% a 57% dos problemas de acessibilidade; testes manuais com leitores de tela continuam indispensáveis.

Performance Web e Lighthouse

PERFORMANCE

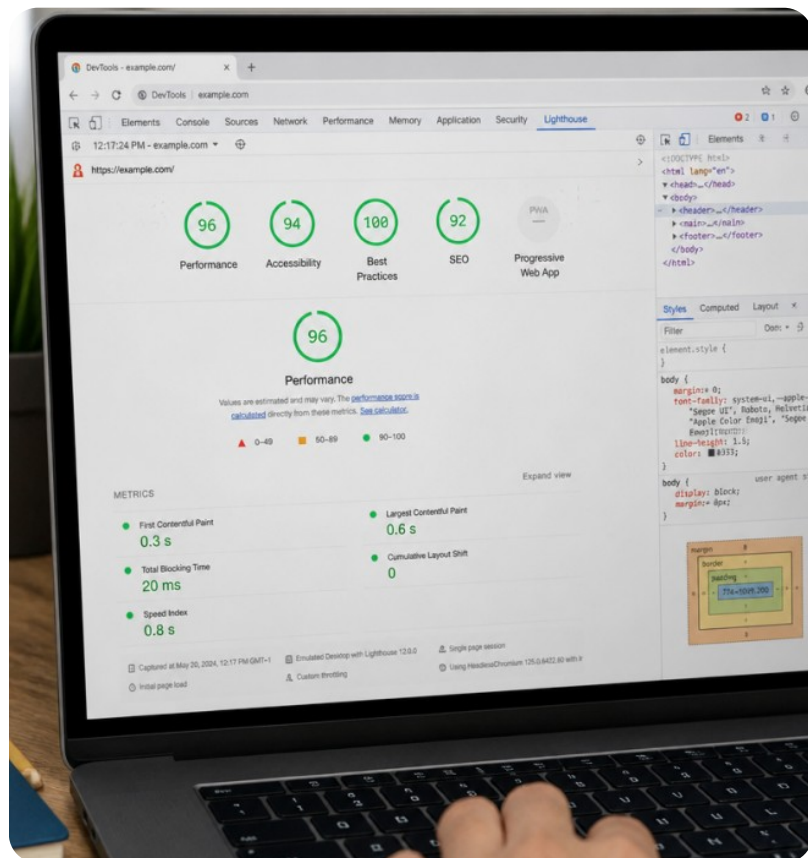
Auditoria de Desempenho

Google Lighthouse audita Performance, Acessibilidade, Boas Práticas e SEO. Executado no Chrome DevTools ou via **Lighthouse CI (LHCI)** em pipelines, gera scores (0-100) e indica otimizações de carregamento.



Ponto-chave

Sites rápidos elevam retenção e UX.



Core Web Vitals Essenciais

LCP

Largest Contentful Paint

Tempo até o maior elemento visual carregar.

Ideal: $\leq 2.5s$.

- Indica velocidade de percepção.

FID / INP

First Input Delay / INP

Tempo de resposta após interação do usuário.

Ideal: FID $\leq 100ms$ ou INP $\leq 200ms$.

- Garante fluidez.

CLS

Cumulative Layout Shift

Estabilidade visual durante o carregamento.

Ideal: pontuação < 0.1 .

- Evita cliques acidentais em elementos móveis.



Lembre-se

Métricas estáveis elevam a satisfação do usuário e o ranqueamento SEO.

Otimizando Pipelines: Paralelização

PERFORMANCE

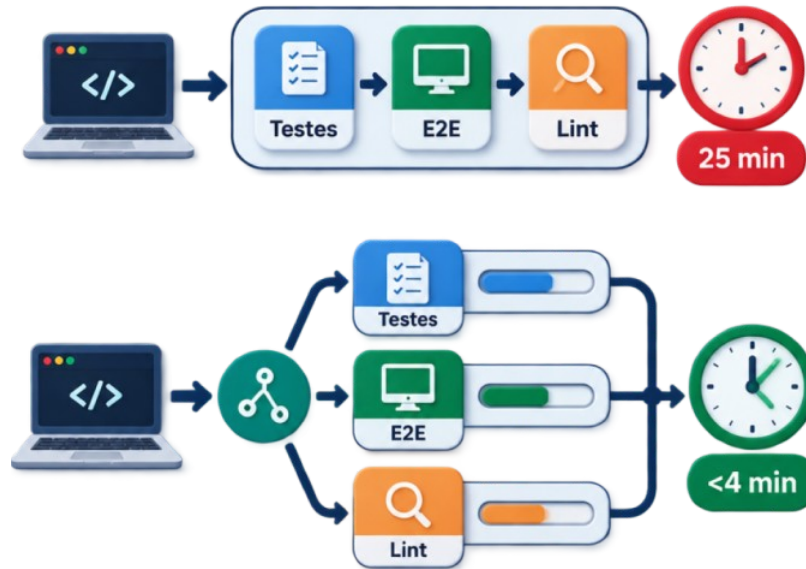
O Problema dos Testes Lentos

Suítes grandes desmotivam commits.
Otimize:

Matriz: Rode testes, E2E e lint em paralelo.

Cache: Salve no cache.

Sharding: Divida testes E2E entre nós.



Curiosidade

Paralelização reduz pipelines de 25min para <4min.

Badges de Status no README

DOCUMENTAÇÃO

Transparência e Confiança

O `exibe` a saúde do projeto via badges:

Build: Pipeline ou .

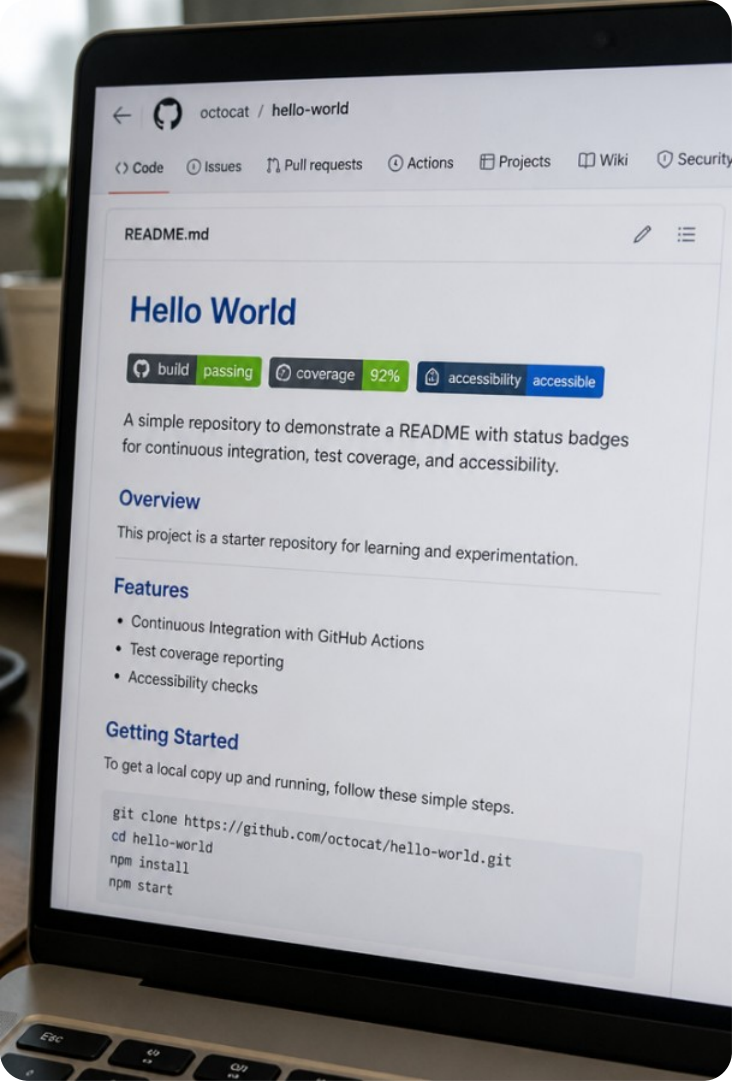
Coverage: % de cobertura (ex: Codecov).

Acessibilidade: Status WCAG via Lighthouse CI.



Exemplo

Sintaxe Markdown:



Sequência do Pipeline de CI

Ordene as etapas lógicas de execução de um pipeline automatizado de testes em um Pull Request:

Execução da suíte de testes unitários, de integração e validação de acessibilidade

Download do código-fonte (actions/checkout) e configuração do Node.js

Instalação limpa e reproduzível das dependências do projeto (npm ci)

Geração do relatório de cobertura e atualização do status de proteção da branch

Sequência do Pipeline de CI

Ordene as etapas lógicas de execução de um pipeline automatizado de testes em um Pull Request:



Download do código-fonte (actions/checkout) e configuração do Node.js

Instalação limpa e reprodutível das dependências do projeto (npm ci)

Execução da suíte de testes unitários, de integração e validação de acessibilidade

Geração do relatório de cobertura e atualização do status de proteção da branch

Atividade Prática 1: Workflow de CI

PRÁTICA GUIADA

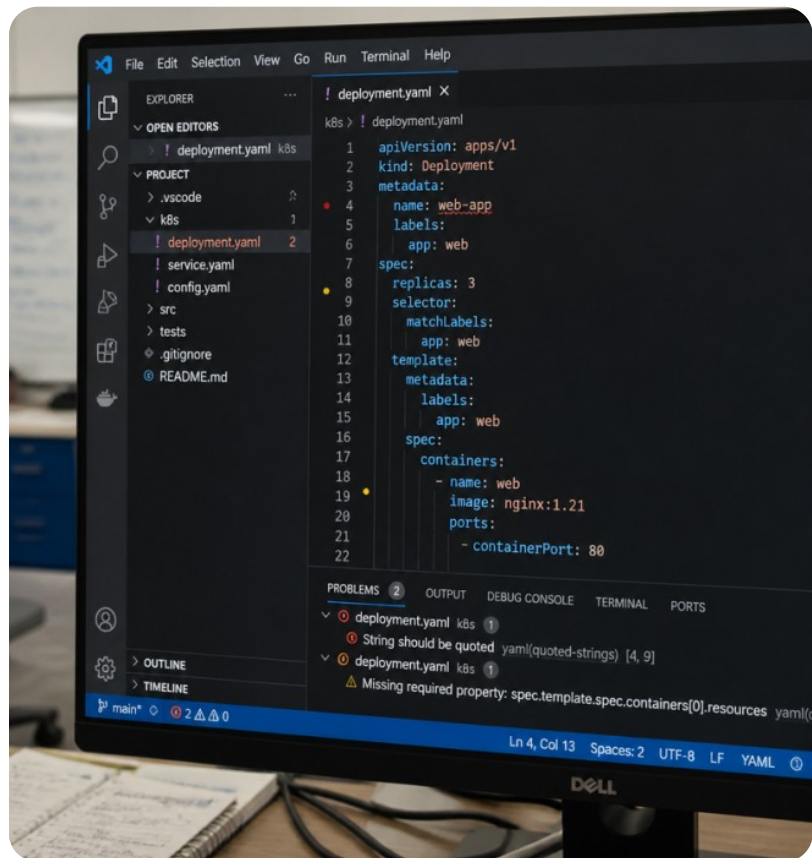
Construindo o Arquivo ci.yml

Crie o arquivo para rodar testes a cada Pull Request. O objetivo é impedir bugs na . Defina um job que baixa o código, instala dependências e executa o Vitest exigindo 80% de cobertura.



Ponto-chave

YAML exige indentação estrita: use dois espaços por nível.



Estrutura do Arquivo ci.yml

Cabeçalho e Gatilho

Dispara automaticamente a cada novo commit ou abertura de PR.

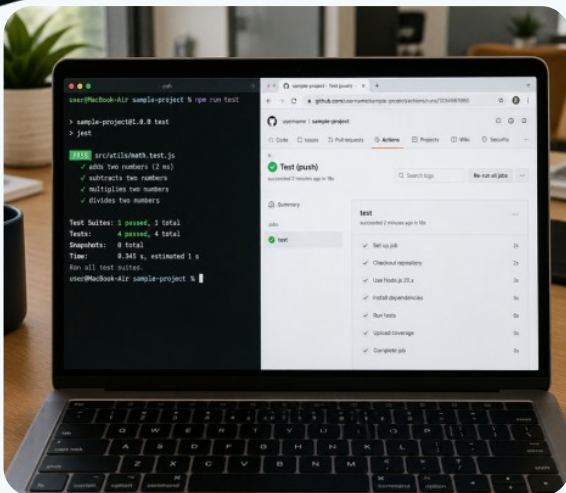
Job e Passos Principais



Lembre-se

O comando `npm run test:coverage` falhará automaticamente se o limite mínimo de cobertura não for atingido.

Desafio Prático: Workflow YAML



Em seu repositório de estudos:

Crie o diretório e o arquivo .

Adicione ao :

1. Abra um Pull Request com um teste quebrado.
1. Verifique o Actions e confirme que o merge foi impedido.
1. Corrija o teste e acompanhe o selo verde ser liberado.

Atividade Prática 2: Auditoria A11y e Performance

PRÁTICA GUIADA

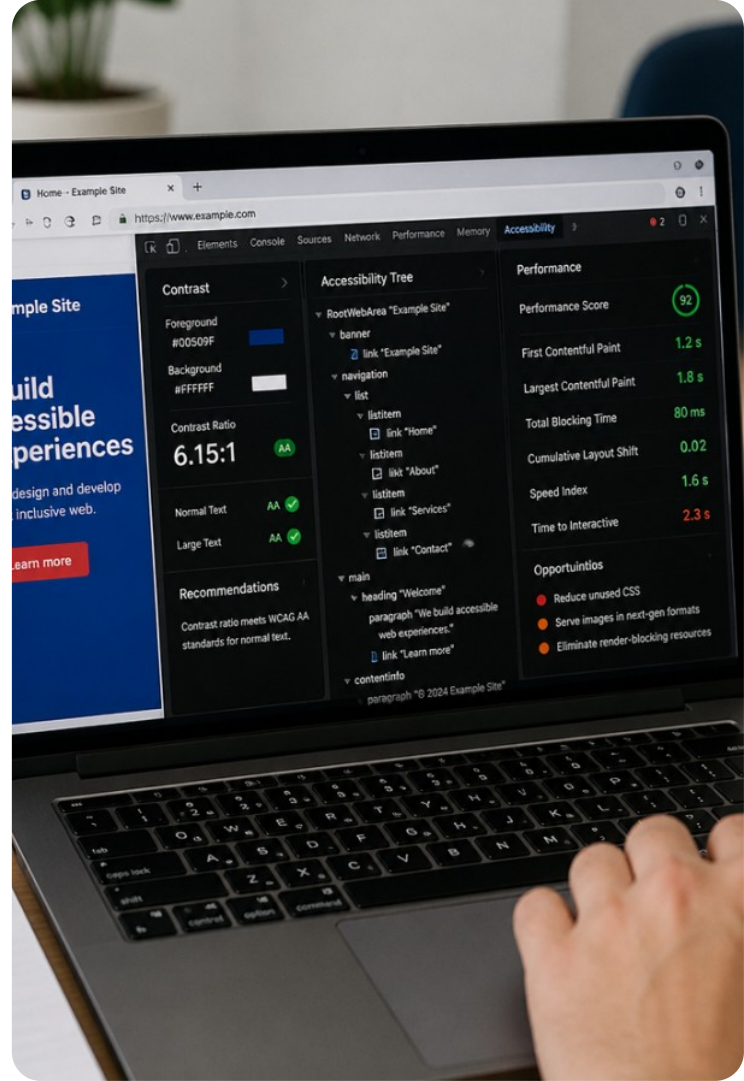
Auditoria: axe-core e Lighthouse

Audite acessibilidade e performance do componente de checkout. Integre ao Vitest e analise métricas Core Web Vitals via Lighthouse (Chrome DevTools).



Ponto-chave

Funcionais validam lógica; não-funcionais validam a experiência do usuário.



Código de Teste de Acessibilidade

Teste com vitest-axe

Configuração de matchers de acessibilidade na suíte.

Execução da Validação

Identifica botões sem rótulo ou inputs sem tags label.



Exemplo

Se um botão contiver apenas um ícone sem aria-label, o teste falhará apontando a regra exata violada.

Desafio Prático: Diagnóstico Real



Etapas no componente de teste:

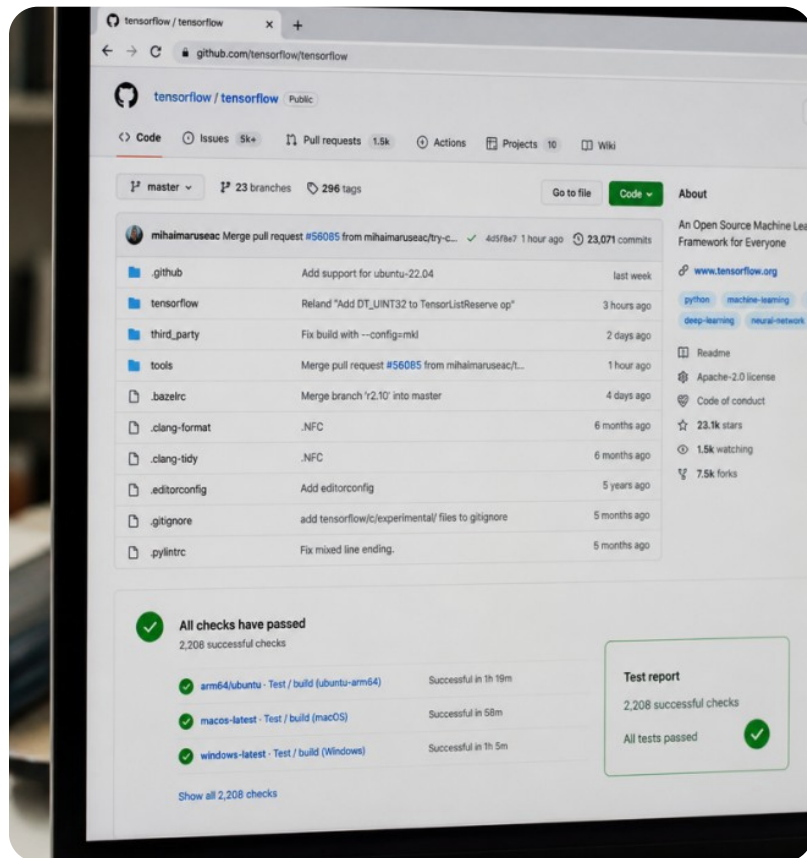
Rode em formulário sem rótulos.

1. Identifique a regra WCAG violada no console.

Adicione e ajuste contraste.

1. Audite com Lighthouse e anote LCP, FID e CLS.

Integrando Badges ao Projeto



DOCUMENTAÇÃO

Montando o README Profissional

Agora que os testes e a pipeline estão operacionais, documentamos a garantia de qualidade diretamente na apresentação do repositório:



Lembre-se

Recrutadores e equipes de tecnologia avaliam repositórios por meio da visibilidade de suas rotinas de teste.

Avaliação: Boas Práticas de CI/CD

Quais das afirmações a seguir representam boas práticas recomendadas para pipelines de testes no frontend? (Mais de uma opção está correta)

1.

Executar testes unitários e auditorias automatizadas a cada Pull Request antes de autorizar o merge.

2.

Configurar cache de dependências e paralelização de jobs para reduzir o tempo da suíte.

3.

Utilizar npm ci em vez de npm install em servidores de CI para garantir builds consistentes.

4.

Desabilitar a execução de testes nos pipelines para acelerar entregas emergenciais sem validação.

Avaliação: Boas Práticas de CI/CD



Quais das afirmações a seguir representam boas práticas recomendadas para pipelines de testes no frontend? (Mais de uma opção está correta)

1.

Executar testes unitários e auditorias automatizadas a cada Pull Request antes de autorizar o merge.

2.

Configurar cache de dependências e paralelização de jobs para reduzir o tempo da suíte.

3.

Utilizar npm ci em vez de npm install em servidores de CI para garantir builds consistentes.

4.

Desabilitar a execução de testes nos pipelines para acelerar entregas emergenciais sem validação.

Síntese dos Quatro Pilares

Engrenagem da Qualidade

Integração Dev-QA:

Funcional: Vitest/Testing Library.

Acessibilidade: axe-core (WCAG).

Performance: Lighthouse/Vitals.

Automação: GitHub Actions (bloqueia falhas/cobertura).



Ponto-chave

Excelência: funcionalidade, acessibilidade, performance e CI/CD.



Preparação para o Projeto Final

O que construímos até aqui

Dominamos Vitest, Testing Library, Cypress, Mocks de API, Cobertura, TDD e Pipelines de CI/CD.

Você possui o conjunto completo de ferramentas utilizado pelas maiores empresas de tecnologia.

Próxima Parada: Bloco 10

Na próxima aula, você desenvolverá o **Projeto Final Integrador**: uma plataforma completa de e-commerce com suíte de testes ponta a ponta e CI/CD configurada na nuvem.



Lembre-se

Mantenha o template do GitHub Actions salvo para acelerar a inicialização do seu projeto capstone.

Debate: Bloquear ou Não Bloquear?



Se um teste de acessibilidade falhar por conta de um contraste leve em um botão secundário, a pipeline de CI deve bloquear completamente o lançamento de uma nova funcionalidade urgente para os clientes? Como a equipe deve ponderar esse trade-off?

Debate: Bloquear ou Não Bloquear?



Você poderia ter dito...

Argumentos pró-bloqueio: Acessibilidade é direito inegociável. Exceções criam dívida técnica difícil de corrigir. Usuários de alto contraste ou leitores de tela são prejudicados.

Argumentos para liberação contingencial: Em crises, priorize a correção imediata no backlog. Mantenha bloqueio estrito e integre acessibilidade desde o design.

Desafio Final de Fixação



Pergunta 1:

Qual métrica Core Web Vitals avalia o tempo de resposta da página quando o usuário interage pela primeira vez?

Pergunta 2:

Em qual diretório do repositório Git os arquivos de workflow do GitHub Actions devem ser obrigatoriamente armazenados?

Pergunta 3:

Qual biblioteca permite integrar auditorias de acessibilidade WCAG diretamente aos testes do Vitest e Testing Library?

Desafio Final de Fixação



Resposta 1:

First Input Delay (FID) ou Interaction to Next Paint (INP).

Resposta 2:

`.github/workflows/`

Resposta 3:

axe-core (ou integrações como vitest-axe / jest-axe).